

Absolutely. Since you're preparing for **Java/Spring Boot backend + distributed systems interviews**, here are scenario-based questions in the same style. These are the kinds of questions where interviewers test **debugging, concurrency, performance, database, Kafka, API resilience, and production thinking** rather than syntax.

30 Production & System-Design Interview Questions with Answers

1. Your API suddenly becomes slow. What do you check first?

Answer:

I would first determine whether the problem is application-wide or limited to a particular endpoint.

I would check:

1. API latency metrics — p50, p95, p99
2. Error rate and traffic volume
3. CPU and memory utilization
4. Database response time
5. Downstream API latency
6. Thread pool utilization
7. Connection pool usage
8. Recent deployments/configuration changes

Then I would use logs, distributed tracing, and metrics to identify which component is consuming the additional time.

Interview keyword: Don't immediately assume the problem is Java code. **Find where the latency is being introduced.**

2. CPU is normal, but response time is high. Where could the bottleneck be?

Answer:

If CPU is normal, I would investigate **I/O and waiting time**.

Possible bottlenecks:

- Slow database queries
- Database connection pool exhaustion
- Slow downstream APIs
- Network latency

- Thread pool exhaustion
- Lock contention
- Kafka delays
- Disk I/O
- DNS/network issues

For example, threads may be sitting idle from the CPU's perspective while waiting for a database response.

3. Two users update the same record simultaneously. How do you prevent data loss?

Answer:

This is a **concurrency control** problem.

Possible approaches include:

Optimistic locking

Add a version column:

```
id | balance | version
1  | 5000    | 3
```

User A reads version **3**.

User B also reads version **3**.

User A updates:

```
version 3 → 4
```

When User B attempts to update version **3**, the update fails because the current version is already **4**.

In JPA, this can be implemented using:

Java

```
@Version
private Long version;
```

For high-contention scenarios, **pessimistic locking** can also be considered.

4. Your application creates too many threads under heavy traffic. What happens?

Answer:

Creating too many threads can cause:

- High memory consumption
- Context switching
- CPU overhead
- Thread scheduling overhead
- Thread pool exhaustion
- Increased latency
- Eventually `OutOfMemoryError` or service instability

I would use a **bounded thread pool** rather than creating unlimited threads.

For example:

```
graph TD
    A[Incoming requests] --> B[Bounded Thread Pool]
    B --> C[Worker Threads]
    C --> D[Task execution]
```

The pool size should be determined based on workload, CPU cores, blocking I/O, and downstream capacity.

5. A Java service keeps consuming memory even after GC. What would you investigate?

Answer:

I would investigate whether there is a **memory leak** or **unexpectedly high live-object retention**.

I would check:

- Heap usage after GC
- GC frequency
- Heap dump

- Dominator tree
- Large collections/maps
- Static references
- Caches
- ThreadLocal usage
- Unclosed resources
- Object retention paths

Tools:

```
JFR
VisualVM
Eclipse MAT
jmap
GC logs
```

The important distinction is:

If GC is working but the **post-GC baseline keeps increasing**, objects are probably still being referenced somewhere.

6. One database query suddenly takes 10 seconds in production. What's your debugging approach?

Answer:

I would compare the production query behavior with normal execution.

I would check:

1. Query execution plan
2. Index availability
3. Table size
4. Statistics
5. Lock contention
6. Database CPU/memory
7. Connection pool
8. Recent schema/data changes
9. Parameter values
10. Whether the query is doing a full table scan

I would use tools such as:

SQL

EXPLAIN

EXPLAIN ANALYZE

I would avoid blindly adding indexes before understanding the execution plan.

7. Your Kafka consumer processes the same message twice. How do you handle it?

Answer:

Kafka consumers can process messages more than once, especially around failures and offset commits.

I would design the consumer to be **idempotent**.

For example, suppose:

```
Transaction ID = TX123
```

Before processing, I check whether `TX123` has already been processed.

```
Kafka message
  ↓
Check transaction ID
  ↓
Already processed?
  /      \
Yes       No
  ↓       ↓
Skip      Process
          ↓
      Store processed ID
```

Depending on the architecture, this can involve:

- Idempotency keys
 - Unique database constraints
 - Transactional processing
 - Appropriate offset management
 - Kafka transactions where applicable
-

8. A third-party API becomes slow. How do you stop it from affecting your service?

Answer:

I would use **resilience patterns**.

Most importantly:

- Timeout
- Circuit breaker
- Retry with exponential backoff
- Retry limits
- Bulkhead isolation
- Fallback response
- Caching where appropriate

For example:

```
Our Service
  ↓
Timeout
  ↓
Third-party API
  ↓
Slow
  ↓
Circuit Breaker
  ↓
Fail fast / fallback
```

A key point is:

Never allow an unbounded downstream call to consume all your application threads.

9. Your application works perfectly in staging but fails in production. What do you compare?

Answer:

I would compare the environments systematically.

Application

- Application version
- Dependencies

- Configuration
- Environment variables
- JVM version
- JVM arguments

Infrastructure

- CPU/memory
- Container limits
- Network
- Load balancer
- Kubernetes configuration

Database

- Schema
- Indexes
- Data volume
- Database version
- Connection pool

External systems

- API endpoints
- Credentials
- Certificates
- Network rules
- Rate limits

I would also compare **actual production logs and traces** instead of relying only on assumptions.

10. A production issue has no obvious exception in the logs. What do you check next?

Answer:

An absence of exceptions doesn't mean the application is healthy.

I would check:

- Request latency
- HTTP status codes
- Access logs

- Distributed traces
- Thread dumps
- GC logs
- Database metrics
- Connection pools
- Kafka lag
- External API latency
- CPU/memory
- Application health metrics

The issue could be **timeout, deadlock, thread starvation, connection exhaustion, or downstream latency** without generating a clear application exception.

More Advanced Questions

11. Your API suddenly receives 10× normal traffic. What would you do?

Answer:

I would first determine whether the traffic is legitimate or abnormal.

Then:

```
Load Balancer
  ↓
Multiple application instances
  ↓
Load distribution
  ↓
Database / downstream services
```

I would consider:

- Horizontal scaling
- Auto-scaling
- Rate limiting
- Caching
- Queue-based asynchronous processing
- Database connection-pool tuning
- Backpressure

But scaling the application alone isn't enough if the database or downstream API has a lower capacity.

12. Your application is responding with HTTP 500 intermittently. How do you debug it?

Answer:

I would correlate the failures using:

Request ID / Trace ID

Then check:

- Stack traces
- Endpoint
- Request parameters
- Instance/pod
- Database calls
- Downstream calls
- Frequency
- Recent deployment

If only one instance produces errors, I would investigate **instance-specific configuration or infrastructure problems**.

13. Your database connection pool is exhausted. What could cause it?

Answer:

Possible causes:

- Too many concurrent requests
- Slow queries
- Connection leaks
- Long-running transactions
- Database unavailable/slow
- Pool size too small

For example:

```
100 requests
  ↓
All waiting for DB connections
  ↓
Connection pool = 20
  ↓
80 requests waiting
```

↓
Latency increases

I would check pool metrics such as **active, idle, and pending connections**.

14. Your application has a deadlock. How would you identify it?

Answer:

I would take a **thread dump**.

In Java, tools such as:

Bash

```
jstack <pid>
```

can help identify threads waiting for locks.

Example:

```
Thread A
  holds Lock 1
    ↓
  waiting for Lock 2

Thread B
  holds Lock 2
    ↓
  waiting for Lock 1
```

Neither can continue.

I would then identify the locking order and redesign the synchronization strategy.

15. Your API has high latency only during peak traffic. What do you investigate?

Answer:

I would look for resource saturation:

- Thread pool
- Database connections
- CPU

- Memory
- GC
- Database
- Downstream APIs
- Kafka
- Network

I would compare:

```
Normal traffic metrics
      vs
Peak traffic metrics
```

Especially **p95/p99 latency**, because averages can hide severe tail latency.

16. How would you prevent one API consumer from overwhelming your service?

Answer:

Use **rate limiting**.

For example:

```
Client
  ↓
Rate Limiter
  ↓
100 requests/sec allowed
  ↓
API
```

Possible algorithms:

- Token Bucket
- Leaky Bucket
- Fixed Window
- Sliding Window

For distributed applications, the rate limiter may need a shared store or gateway-level implementation.

17. Your API calls three downstream services sequentially. How can you reduce latency?

Answer:

If the calls are independent, execute them **in parallel**.

Sequential:

```
API
  ↓
A (200ms)
  ↓
B (300ms)
  ↓
C (200ms)

≈ 700ms
```

Parallel:

```
      ↗ A (200ms)
API ———|→ B (300ms)
      ↘ C (200ms)

≈ 300ms
```

In Java, depending on the application architecture, this could be implemented using:

Java

```
CompletableFuture
```

or an appropriate asynchronous/reactive approach.

But I would also consider downstream rate limits and thread-pool capacity before blindly parallelizing calls.

18. Your downstream API allows only 10 TPS, but your clients send 100 TPS. What would you do?

Answer:

I would not simply forward all 100 requests.

I would introduce **rate limiting, batching, caching, or asynchronous processing**, depending on the business requirement.

For example:

```
100 TPS Client
    ↓
Queue / Buffer
    ↓
Rate Limiter
    ↓
10 TPS
    ↓
Downstream API
```

If the downstream API supports batching, I could combine multiple requests into a single call.

If data is relatively stable, caching can reduce downstream calls significantly.

19. Your API needs to handle a downstream timeout. What should happen?

Answer:

I would configure a bounded timeout.

For example:

```
Request
    ↓
Downstream API
    ↓
Timeout
    ↓
Fallback / meaningful error
```

I would avoid allowing the request to wait indefinitely.

Depending on the operation, I might return:

```
HTTP 504 Gateway Timeout
```

or a business-specific response.

20. Your application has a memory spike every few hours. What would you investigate?

Answer:

I would look for periodic activities:

- Scheduled jobs
- Batch processing
- Cache refresh
- Large file processing
- Large database queries
- Kafka batch consumption
- Heap growth
- Temporary object creation

I would correlate the memory graph with application events.

Then take a **heap dump during/after the spike** and analyze retained objects.

Java-Specific Scenario Questions

21. HashMap suddenly becomes slow. What could be wrong?

Answer:

I would investigate:

- Poor hash distribution
- Excessive collisions
- Large map size
- Incorrect `equals()` / `hashCode()` implementation
- Concurrent modification/use
- Excessive resizing

For custom objects used as keys:

Java

```
@Override
public boolean equals(Object obj) { ... }

@Override
public int hashCode() { ... }
```

must follow the Java contract.

If:

```
a.equals(b) == true
```

then:

```
a.hashCode() == b.hashCode()
```

must also be true.

22. Why would you choose ConcurrentHashMap instead of HashMap?

Answer:

`HashMap` is not designed for concurrent modification.

`ConcurrentHashMap` provides thread-safe concurrent access with much better scalability than synchronizing an entire map.

I would choose it when multiple threads need to access and modify shared map data concurrently.

23. Your Java application has high GC activity. What would you check?

Answer:

I would investigate:

- Allocation rate
- Heap size
- Young/old generation behavior
- GC frequency
- GC pause duration
- Object lifetime
- Large object allocations
- Memory leaks
- Cache growth

I would use:

```
JFR
GC logs
Heap dump
VisualVM
```

The goal is to determine whether the problem is **excessive allocation**, **insufficient heap**, **inefficient GC configuration**, or **object retention**.

24. One endpoint is slow, but all other endpoints are fast. What does that tell you?

Answer:

It suggests the problem is likely **localized to that endpoint or one of its dependencies**.

I would investigate:

```
Endpoint
  ↓
Business logic
  ↓
Database queries
  ↓
Downstream APIs
  ↓
Serialization
```

I would compare its trace with a fast endpoint and identify the slow span.

25. Your service suddenly starts returning `OutOfMemoryError`. What do you do?

Answer:

First, I would determine which memory area is exhausted.

Possibilities include:

```
Java heap
Metaspace
Direct/off-heap memory
Native memory
```

For heap exhaustion, I would capture/analyze a heap dump and inspect:

- Largest objects
- Collections
- Caches
- Static references
- Object retention

I would also check whether the issue started after a recent deployment.

26. How would you design an API so that a duplicate request doesn't create duplicate transactions?

Answer:

I would use **idempotency**.

The client sends:

```
Idempotency-Key: ABC123
```

The server stores the result associated with that key.

```
Request ABC123
  ↓
Already processed?
  /      \
Yes       No
  ↓       ↓
Return    Process
existing  ↓
result    Store result
```

This is particularly important for operations such as:

```
Payment
Transfer
Order creation
Account transaction
```

27. A Kafka consumer is falling behind. What would you check?

Answer:

I would check **consumer lag** first.

Then investigate:

- Number of partitions
- Number of consumers
- Consumer processing time
- Database latency
- Downstream API latency
- Consumer errors/retries
- Poll configuration
- Rebalancing
- CPU/memory

If processing is the bottleneck, I might increase consumer parallelism **within the partitioning constraints** or optimize processing.

28. Your service receives a request but the downstream service is unavailable. Should you retry?

Answer:

Not blindly.

Retry depends on:

- Type of failure
- Operation idempotency
- Retry count
- Timeout
- Backoff
- Downstream capacity

For transient failures:

```
Attempt 1
  ↓
Failure
  ↓
Backoff
  ↓
Attempt 2
  ↓
Failure
  ↓
Backoff
```

I would use **exponential backoff with jitter** and a maximum retry count.

For non-transient errors such as validation failures, retrying is usually pointless.

29. Your database is healthy, but application requests are timing out. What could be happening?

Answer:

I would investigate components between the application and database:

```
Client
  ↓
Load Balancer
  ↓
Application
  ↓
Connection Pool
  ↓
Database
```

Possible issues:

- Connection pool exhaustion
- Thread pool exhaustion
- Network latency
- Application locks
- Slow serialization
- Downstream API calls
- Application-level timeout
- Load balancer timeout

A healthy database doesn't necessarily mean the application can successfully obtain and use a database connection.

30. You deploy a new version and latency immediately increases. What would you do?

Answer:

I would first correlate the timing:

Deployment



Latency increase

Then compare the old and new versions.

I would investigate:

- New database queries
- Additional downstream calls
- New logging
- Configuration changes
- Thread pools
- Connection pools
- Serialization
- GC behavior
- New code paths

If the impact is significant and unexplained, I would **rollback or disable the problematic feature** first, then perform deeper analysis.

★ 10 Questions You Should Especially Prepare

Given your **Java + REST API + downstream API + Agentic AI/MCP** background, I would prioritize these for interviews:

1. **How do you handle downstream API rate limits?**
2. **How do you handle 100 TPS incoming traffic when downstream supports only 10 TPS?**
3. **How do you implement caching?**
4. **How do you prevent duplicate transactions?**
5. **How do you handle concurrent updates to the same database record?**
6. **How does a circuit breaker work?**
7. **How do you debug high API latency?**
8. **How do you handle Kafka duplicate messages?**
9. **How do you troubleshoot Java memory leaks?**
10. **How would you design an API that calls multiple downstream services efficiently?**

These questions are particularly useful because they test whether you understand **how a backend system behaves in production**, rather than just whether you know Java syntax.